

BINARY SEARCH TREE IMPLEMENTATION AND ANALYSIS

-REPORT SUMMARY

Author: Van Robbins

Course: C310

Assignment: Project 2

Date: June 22, 2025

IMPLEMENTATION OVERVIEW

1. BST CLASS IMPLEMENTATION

TreeNode class for individual nodes

BinarySearchTree class with all required methods:

- insert(value): $O(\log n)$ average, $O(n)$ worst case
- delete(value): $O(\log n)$ average, $O(n)$ worst case
- search(value): $O(\log n)$ average, $O(n)$ worst case
- in_order_traversal(): $O(n)$ - returns sorted order
- pre_order_traversal(): $O(n)$ - root-left-right
- post_order_traversal(): $O(n)$ - left-right-root
- height(): $O(n)$ - calculates tree height

2. KEY FEATURES

Recursive implementation for all operations

Proper handling of edge cases (empty tree, duplicates) OK Memory efficient

node structure

Type hints for better code documentation

EXPERIMENTAL RESULTS

TASK 2: 64 Random Integers [1, 128]

- Generated values: [29, 4, 95, 36, 32, 128, 18, 126, 14, 87...]
- Tree height after insertions: 11
- Expected balanced height: ~ 7
- Height efficiency: 63.6%
- Insertion time: 0.0000 seconds
- All search operations successful (first, middle, last values)
- All non-existent searches correctly returned false
- Traversals verified (in-order produces sorted sequence)

- Successful deletions with BST property maintained

TASK 3: Different Dataset Results

RANDOM DATA PERFORMANCE:

Dataset Size	Range	Height	Expected	Efficiency	Performance
64	[1,128]	11	7	63.6%	Good
128	[1,256]	16	8	50.0%	Fair
256	[1,512]	18	9	50.0%	Fair

WORST-CASE SCENARIO RESULTS

ASCENDING ORDER PERFORMANCE:

Size	Height	Expected	Efficiency	Performance Impact
64	64	7	10.9%	9.1x worse height
128	128	8	6.2%	16.0x worse height
256	256	9	3.5%	28.4x worse height

ADDITIONAL SCENARIOS:

- Descending Order (32 values): Height 32, creates right-heavy tree
- Alternating Pattern (32 values): Height 32, still unbalanced
- Duplicate Values (20 identical): Height 1, only one node stored

KEY FINDINGS

1. RANDOM DATA PERFORMANCE

- Heights stay reasonably close to $O(\log n)$ for small datasets
- Efficiency decreases as dataset size increases (63.6% → 50.0%)
- Natural randomization creates moderately balanced trees
- All operations remain efficient for practical purposes

2. WORST-CASE DEGRADATION

- Ascending/descending insertion creates linked-list structure
- Height becomes $O(n)$ instead of $O(\log n)$
- Dramatic performance degradation (9x to 28x worse)
- Search performance severely impacted in large datasets

3. TRAVERSAL VERIFICATION

- In-order traversal always produces sorted output OK
- Pre-order useful for tree reconstruction OK
- Post-order useful for deletion operations OK
- All traversals maintain correct node count OK

4. OPERATION PERFORMANCE

- Insertion times: < 0.01 seconds for all test cases
- Search times: < 0.000001 seconds for all cases
- Deletion maintains BST property correctly
- Duplicate values properly ignored (no duplicates stored)

5. HEIGHT ANALYSIS TRENDS

- Random data: Height $\sim 1.6-2.0 \times \log_2(n)$
- Worst case: Height = n (complete degradation)
- Performance directly correlates with tree height
- Efficiency drops significantly with larger datasets

COMPARATIVE ANALYSIS

AVERAGE CASE (Random Data):

- Time Complexity: $O(\log n)$ for search, insert, delete
- Space Complexity: $O(n)$ for storage
- Height: $\sim 1.6-2.0 \times \log_2(n)$ typically
- Suitable for most practical applications
- Performance degrades gradually with size

WORST CASE (Sorted Data):

- Time Complexity: $O(n)$ for all operations
- Space Complexity: $O(n)$ for storage
- Height: n (essentially a linked list)
- Unacceptable for large datasets
- Complete performance breakdown

PRACTICAL IMPLICATIONS

1. INPUT DATA MANAGEMENT

- Randomize input when possible to avoid worst-case
- Avoid pre-sorted data insertion patterns
- Monitor tree height in production systems
- Consider data preprocessing for better balance

2. ALGORITHM IMPROVEMENTS

- Self-balancing trees (AVL, Red-Black) recommended for production

- Current implementation suitable for educational purposes
- Consider rotation operations for guaranteed $O(\log n)$ performance
- Hybrid approaches may be needed for specific use cases

3. PERFORMANCE EXPECTATIONS

- Small datasets (≤ 64): Generally acceptable performance
- Medium datasets (128-256): Noticeable height increase
- Large datasets: Require self-balancing for reliability
- Worst-case scenarios: Completely unacceptable performance

CONCLUSION

The experimental results provide compelling evidence for why advanced tree structures with guaranteed balance are preferred in real-world applications, while simple BSTs remain valuable for educational purposes and specific use cases with naturally random data.